

COMPUTE, STORAGE, NETWORKING AND THE DEPLOYMENT LANDSCAPE

Lab Guide

Six hands-on labs and two activities on Google Cloud — console or Cloud Shell only, no code to write.

Version 2.0 · September 2026 · gcloud 583.x · region asia-south1

Contents

How to use this guide	3
Lab 0 — Landing in the sandbox.....	4
Lab 1 — A VM, then a fleet	6
Lab 2 — A cluster, a deployment, a service.....	10
Lab 3 — An HTTP function, then a function that reacts to an upload.....	14
Lab 4 — A private subnet that can still reach Google	17
Lab 5 — Postgres from psql, then Firestore documents	21
Activities A and B (no keyboard)	25
Optional — a catalogue bucket that cleans up after itself	27
Appendix — Environment and versions	30

How to use this guide

Every lab in this guide is done either in the Google Cloud console or with commands typed into Cloud Shell. You never write or edit application code: where a lab deploys an application, the application is supplied complete in the lab kit and your job is to build, store, deploy, secure and observe it.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order.
- Blanks written as _____ in a starter script are the parts you fill in; the answers are in the expected panels here and on the debrief slides.
- Every step that has a console equivalent shows a Console path line: the navigation, the page and the button. Use whichever you prefer — the result is identical.
- Expected panels show the real output format. Names, IPs, timestamps and hashes will differ in your project.
- Angle-bracket placeholders such as <your-project-id> must be replaced with your own value. The scripts read the project id for you with `gcloud config get-value project`.
- Every lab that creates resources ends with a teardown step. A lab is finished when the resource list is empty — not when the page loads.

Getting the kit into Cloud Shell

Upload the zip through the Cloud Shell menu (: → Upload) or drag it onto the terminal, then:

```
$ unzip -q gcp-landscape-lab1-starter.zip && cd lab1 && ls
README.md lab1.sh startup.sh teardown.sh
```

Where to get help

- Raise a hand — the trainer pairs you with a neighbour rather than debugging in front of the room.
- Each lab ends with a Troubleshooting section covering the failures that actually happen.
- The solution kit for each lab is the completed command sheet; the trainer releases it after the debrief.

Cost note

Everything here runs inside a sandbox project the programme pays for. Idle resources still cost money overnight — three small VMs are roughly US\$1 a day, an idle GKE Autopilot cluster US\$2–3 a day. Tear down before every break.

Lab 0 — Landing in the sandbox

Estimated time	Difficulty	Environment	Kit
10 min	●○○	Cloud Shell, region asia-south1	none

OBJECTIVE

Cloud Shell open on your sandbox project, gcloud pointed at asia-south1, the six APIs enabled and the toolchain verified.

PREREQUISITES

- A Google account added to a sandbox project as Owner (sent as pre-work).
- A browser. Nothing to install.

Steps

Step 1 — Open Cloud Shell on your sandbox project

Console path: console.cloud.google.com → project picker (top bar) → your sandbox → Activate Cloud Shell icon (top right).

EXPECTED

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to sandbox-rohan-07.
rohan@cloudshell:~ (sandbox-rohan-07) $
```

Step 2 — Point gcloud at the project, region and zone

CLOUD SHELL

```
gcloud config set project <your-project-id>
gcloud config set compute/region asia-south1
gcloud config set compute/zone asia-south1-a
gcloud config list
```

EXPECTED

```
Updated property [core/project].
Updated property [compute/region].
Updated property [compute/zone].
[compute]
region = asia-south1
zone = asia-south1-a
[core]
account = you@example.com
project = sandbox-rohan-07
```

Step 3 — Enable the six APIs used today

Console path: APIs & Services → Enable APIs and services → search each API → Enable (six times). The command does all six at once.

CLOUD SHELL

```
gcloud services enable compute.googleapis.com run.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com \
  secretmanager.googleapis.com storage.googleapis.com
```

EXPECTED

```
Operation "operations/acat.p2-6..." finished successfully.
```

Takes 30–90 seconds. Re-running is harmless.

Step 4 — Verify the toolchain

CLOUD SHELL

```
gcloud --version | head -1
docker --version
python3 --version
kubectl version --client | head -1
```

EXPECTED

```
Google Cloud SDK 583.0.0
Docker version 28.x
Python 3.13.5
Client Version: v1.33.x
```

CHECK YOURSELF

- The prompt ends with (your-project-id).
- `gcloud config list` shows `asia-south1 / asia-south1-a`.
- `gcloud services list --enabled | grep -c googleapis` prints a number ≥ 6 .

Troubleshooting

Symptom	Cause	Fix
"You do not currently have an active account selected"	Cloud Shell opened in the wrong Google account	Switch account (avatar, top right) and reopen Cloud Shell
Enable fails with <code>PERMISSION_DENIED</code>	You are not Owner on this project	Tell the trainer — you get the spare project
Cloud Shell stuck on "Provisioning"	Session start-up hiccup	Reload the browser tab; if it persists, pair up
Prompt shows another project	Several sandboxes in the picker	<code>gcloud config set project <your-project-id></code>

Summary

You now have one project, one region and six enabled APIs. Every later lab assumes exactly this state — if you switch machines or accounts, redo steps 2 and 3.

Lab 1 — A VM, then a fleet

Estimated time	Difficulty	Environment	Kit
20 min	●●○	Cloud Shell, region asia-south1	gcp-landscape-lab1-starter.zip (solution: -solution.zip)

OBJECTIVE

One Compute Engine VM serving a page from a startup script, then an instance template, a two-VM managed instance group and an autoscaler — and everything torn down.

PREREQUISITES

- Lab 0 completed.
- Kit unzipped in Cloud Shell: `unzip gcp-landscape-lab1-starter.zip && cd lab1` — you get `lab1.sh` (three blanks), `startup.sh`, `teardown.sh` and `README.md`.

The startup script

`startup.sh` ships in the kit (`lab1/startup.sh`). It is not code you write — it is the recipe the VM runs as root at every boot: install Python, write one HTML page, serve it on port 80. The Cloud Shell steps pass it by file name; on the console route you paste its content into Create instance → Advanced options → Management → Automation → Startup script.

LAB1/STARTUP.SH — SUPPLIED IN THE KIT

```
#!/bin/bash
# Startup script for the Bookshelf baseline VM / MIG template.
# Serves one page on port 80 that says which VM answered.
apt-get update -y && apt-get install -y python3
mkdir -p /srv/www
cat > /srv/www/index.html <<HTML
<h1>Bookshelf baseline</h1>
<p>Served by VM: $(hostname)</p>
HTML
cd /srv/www && nohup python3 -m http.server 80 >/var/log/bookshelf.log 2>&1 &
```

Steps

Step 1 — Create one VM from the debian-12 image family with the startup script

Console path: Compute Engine → VM instances → Create instance → name bookshelf-vm, region asia-south1, zone a, e2-micro, Boot disk: Debian 12, Networking: tag http-server, Advanced options → Management → Automation → Startup script: paste the content of `startup.sh` (printed above).

CLOUD SHELL

```
gcloud compute instances create bookshelf-vm \
  --machine-type=e2-micro \
  --image-family=debian-12 --image-project=debian-cloud \
  --boot-disk-size=10GB --boot-disk-type=pd-balanced \
  --tags=http-server \
  --metadata-from-file=startup-script=startup.sh
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/instances/bookshelf-vm].
NAME          ZONE          MACHINE_TYPE  PREEMPTIBLE  INTERNAL_IP  EXTERNAL_IP
STATUS
bookshelf-vm  asia-south1-a  e2-micro      10.160.0.2   34.93.118.204
RUNNING
```

Step 2 — Allow HTTP to anything tagged http-server

Console path: VPC network → Firewall → Create firewall rule → name allow-http-80, network default, ingress, allow, targets: specified tags http-server, source 0.0.0.0/0, tcp:80.

CLOUD SHELL

```
gcloud compute firewall-rules create allow-http-80 \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --source-ranges=0.0.0.0/0 --target-tags=http-server
```

EXPECTED

Creating firewall...done.

NAME	NETWORK	DIRECTION	PRIORITY	ALLOW	DENY	DISABLED
allow-http-80	default	INGRESS	1000	tcp:80		False

Step 3 — Look at the VM and open its page**CLOUD SHELL**

```
gcloud compute instances list
gcloud compute instances describe bookshelf-vm \
  --format="value(networkInterfaces[0].accessConfigs[0].natIP)"
```

EXPECTED

34.93.118.204

Open <http://<that IP>> in a browser tab. Within ~60 s of boot you should see "Bookshelf baseline — Served by VM: bookshelf-vm".

CHECK YOURSELF

- The page loads and names bookshelf-vm.
- If it does not: wait a minute (apt-get is installing python3), then check the tag on the VM matches the firewall rule.

Step 4 — Fill blank 1.4 — an instance template with the same recipe, e2-small

Console path: Compute Engine → Instance templates → Create instance template → same choices as the VM, machine type e2-small.

CLOUD SHELL

```
gcloud compute instance-templates create bookshelf-tmpl \
  --machine-type=e2-small \
  --image-family=debian-12 --image-project=debian-cloud \
  --tags=http-server \
  --metadata-from-file=startup-script=startup.sh
```

EXPECTED

Created [<https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/global/instanceTemplates/bookshelf-tmpl>].

NAME	MACHINE TYPE	PREEMPTIBLE	CREATION TIMESTAMP
bookshelf-tmpl	e2-small		2026-09-06T10:31:42.117+05:30

Reference the image family, never a dated image name — next month's VMs get this month's patches for free.

Step 5 — Fill blank 1.5 — a zonal managed instance group of 2

Console path: Compute Engine → Instance groups → Create instance group → New managed instance group (stateless) → template bookshelf-tmpl, single zone asia-south1-a, number of instances 2.

CLOUD SHELL

```
gcloud compute instance-groups managed create bookshelf-mig \
```

```
--template=bookshelf-tmpl --size=2 --zone=asia-south1-a
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/instanceGroupManagers/bookshelf-mig].
NAME          LOCATION    SCOPE  BASE_INSTANCE_NAME  SIZE  TARGET_SIZE
INSTANCE_TEMPLATE  AUTOSCALED
bookshelf-mig  asia-south1-a  zone   bookshelf-mig       0     2           bookshelf-
tmpl          no
```

Step 6 — Fill blank 1.6 — autoscale 1..3 on 60 % CPU with a 60 s cooldown

Console path: Instance groups → bookshelf-mig → Edit → Autoscaling: On, min 1, max 3, signal CPU utilisation 60 %, cool-down 60 s.

CLOUD SHELL

```
gcloud compute instance-groups managed set-autoscaling bookshelf-mig \
  --zone=asia-south1-a --min-num-replicas=1 --max-num-replicas=3 \
  --target-cpu-utilization=0.6 --cool-down-period=60
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/autoscalers/bookshelf-mig-xkq2].
---
autoscalingPolicy:
  coolDownPeriodSec: 60
  cpuUtilization: {utilizationTarget: 0.6}
  maxNumReplicas: 3
  minNumReplicas: 1
  mode: ON
```

Step 7 — Watch the group

Console path: Instance groups → bookshelf-mig → Monitoring tab shows the autoscaler decisions.

CLOUD SHELL

```
gcloud compute instance-groups managed list-instances bookshelf-mig --zone=asia-south1-a
```

EXPECTED

NAME	ZONE	STATUS	ACTION	INSTANCE_TEMPLATE
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE	bookshelf-tmpl
bookshelf-mig-x2mc	asia-south1-a	RUNNING	NONE	bookshelf-tmpl
# about 10 minutes later, VMs idle, min=1:				
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE	bookshelf-tmpl

Predict before you look: two idle VMs, minimum 1 — the autoscaler will shrink the group to one after its stabilisation window.

Step 8 — Tear down

CLOUD SHELL

```
bash teardown.sh
gcloud compute instances list # must be empty
```

EXPECTED

```
Lab 1 resources removed.
Listed 0 items.
```

CHECK YOURSELF

- list-instances showed two RUNNING VMs from bookshelf-tmpl.
- The autoscaler shows mode ON with min 1 / max 3.
- After teardown, instances list is empty and the firewall rule is gone.

Troubleshooting

Symptom	Cause	Fix
Page never loads	Firewall tag mismatch, or startup script still running	Check --tags=http-server on the VM; wait 60 s; gcloud compute ssh bookshelf-vm --command='tail /var/log/bookshelf.log'
QUOTA_EXCEEDED on create	Sandbox CPU quota	Use e2-small everywhere and size 1 for the MIG
set-autoscaling: "not found"	Zone flag missing or wrong	Always pass --zone=asia-south1-a for zonal groups
Subnet/firewall delete fails in teardown	Order — VMs first	Re-run teardown.sh; it is idempotent

Challenge (fast finishers)

Make the group regional: recreate it with --region=asia-south1 instead of --zone and list where the VMs landed. Then attach a health check (gcloud compute health-checks create http hc-80 --port=80) and enable autohealing with update --health-check=hc-80 --initial-delay=120.

Summary

You built the left edge of the compute spectrum: a VM from an image family, a reusable template, a managed group that replaces unhealthy members, and an autoscaler that changed the group size on its own. The concept it demonstrates: never build the second VM by hand.

Lab 2 — A cluster, a deployment, a service

Estimated time	Difficulty	Environment	Kit
25 min (+ 5–7 min cluster creation at kick-off)	●●○	Cloud Shell, region asia-south1	gcp-landscape-lab2-gke.zip

OBJECTIVE

A GKE Autopilot cluster running three copies of a sample web app behind a LoadBalancer Service, built imperatively and then confirmed from a supplied manifest — and deleted.

PREREQUISITES

- Lab 0 completed.
- The cluster creation started at the chapter kick-off (Step 0 below).
- Kit unzipped: lab2/deployment.yaml (supplied; read and apply, do not edit).

Steps

Step 0 — Kick-off (done at the start of the chapter) — create the Autopilot cluster

Takes five to seven minutes. Leave it running in one Cloud Shell tab and open a second tab (the + icon) for the concept slides.

Console path: Kubernetes Engine → Clusters → Create → Autopilot → name hello-cluster, region asia-south1 → Create.

CLOUD SHELL

```
gcloud services enable container.googleapis.com
gcloud container clusters create-auto hello-cluster --location asia-south1
```

EXPECTED

```
Creating cluster hello-cluster in asia-south1... done.
kubeconfig entry generated for hello-cluster.
NAME          LOCATION    MASTER_VERSION  MASTER_IP      MACHINE_TYPE    NODE_VERSION
hello-cluster  asia-south1  1.33.4-gke.1134000  34.93.201.17   e2-small        1.33.4-gke.1134000
RUNNING
```

Step 1 — Connect kubectl to the cluster

CLOUD SHELL

```
gcloud container clusters get-credentials hello-cluster --location asia-south1
kubectl get nodes
```

EXPECTED

```
Fetching cluster endpoint and auth data.
kubeconfig entry generated for hello-cluster.
No resources found          # normal on Autopilot: nodes appear when pods need them
```

Step 2 — Create a Deployment from Google's sample image

Console path: Kubernetes Engine → Workloads → Deploy → Existing container image: us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0 → name hello-web → Deploy.

CLOUD SHELL

```
kubectl create deployment hello-web --image=us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0
kubectl get pods
```

EXPECTED

```
deployment.apps/hello-web created
NAME                READY   STATUS    RESTARTS   AGE
hello-web-6c9f8d7b4-2kx9p   1/1     Running   0           40s
```

The first pod can take a minute: Autopilot provisions a node for it.

Step 3 — Expose it with a LoadBalancer Service and wait for the external IP

Console path: Workloads → hello-web → Expose → Service type Load balancer, port 80, target port 8080 → Expose. Then Services & Ingress → hello-web.

CLOUD SHELL

```
kubectl expose deployment hello-web --type LoadBalancer --port 80 --target-port 8080
kubectl get service hello-web -w      # Ctrl-C once EXTERNAL-IP is no longer <pending>
curl -s http://<EXTERNAL-IP>/
```

EXPECTED

```
service/hello-web exposed
NAME                TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
hello-web           LoadBalancer  34.118.224.9 <pending>      80:31877/TCP     12s
hello-web           LoadBalancer  34.118.224.9 34.93.156.203 80:31877/TCP     95s
Hello, world!
Version: 2.0.0
Hostname: hello-web-6c9f8d7b4-2kx9p
```

1–2 minutes for the IP. Open it in a browser too; refresh after step 4 to see different hostnames.

CHECK YOURSELF

- One pod Running.
- The Service has an EXTERNAL-IP and curl prints Hello, world! with a Hostname.

Step 4 — Scale to three and watch pods and nodes

Console path: Workloads → hello-web → Actions → Scale → 3.

CLOUD SHELL

```
kubectl scale deployment hello-web --replicas 3
kubectl get pods -o wide
kubectl get nodes
```

EXPECTED

```
deployment.apps/hello-web scaled
NAME                READY   STATUS    RESTARTS   AGE   IP           NODE
hello-web-6c9f8d7b4-2kx9p   1/1     Running   0           4m    10.68.0.12   gk3-
hello-cluster-nap-1p2q...   1/1     Running   0           21s   10.68.1.4    gk3-
hello-web-6c9f8d7b4-7hrmz   1/1     Running   0           21s   10.68.0.11   gk3-
hello-cluster-nap-8r3t...   1/1     Running   0           21s   10.68.0.11   gk3-
hello-web-6c9f8d7b4-m14wc   1/1     Running   0           21s   10.68.0.11   gk3-
hello-cluster-nap-1p2q...
```

Step 5 — Self-healing — delete a pod by hand and watch it come back

CLOUD SHELL

```
kubectl delete pod <one pod name from the list>
sleep 5; kubectl get pods
```

EXPECTED

```
pod "hello-web-6c9f8d7b4-2kx9p" deleted
NAME                READY   STATUS    RESTARTS   AGE
```

```
hello-web-6c9f8d7b4-7hrmz    1/1    Running    0        2m
hello-web-6c9f8d7b4-m14wc    1/1    Running    0        2m
hello-web-6c9f8d7b4-x4pq2    1/1    Running    0        4s    # new name: the
Deployment filled the gap
```

Step 6 — Declarative — read the supplied manifest, apply it, confirm nothing changed

deployment.yaml is the same Deployment (3 replicas) and Service written down, plus resource requests (Autopilot bills what you request).

Console path: Kubernetes Engine → Workloads → hello-web → YAML tab shows the live object in the same form.

CLOUD SHELL

```
unzip -q gcp-landscape-lab2-gke.zip && cat lab2/deployment.yaml
kubectl apply -f lab2/deployment.yaml
kubectl get deployment hello-web
```

EXPECTED

```
deployment.apps/hello-web configured
service/hello-web unchanged
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
hello-web     3/3      3             3            9m
```

Step 7 — Teardown — the Service, then the cluster

The cluster bills its control plane while idle; this is the most expensive thing of the day to forget.

Console path: Kubernetes Engine → Clusters → hello-cluster → Delete.

CLOUD SHELL

```
kubectl delete service hello-web
gcloud container clusters delete hello-cluster --location asia-south1 --quiet
gcloud container clusters list    # must be empty
```

EXPECTED

```
service "hello-web" deleted
Deleting cluster hello-cluster...done.
Listed 0 items.
```

Troubleshooting

Symptom	Cause	Fix
EXTERNAL-IP stays <pending> > 3 min	Load balancer provisioning	Keep watching; check Services & Ingress in the console for an event
kubectl: connection refused	Credentials not fetched or wrong project	Re-run get-credentials; gcloud config get-value project
Pods Pending for minutes	Autopilot provisioning a node	kubectl describe pod <name> shows the scheduling events; wait
create-auto quota error	Sandbox CPU/SSD quota	Pair up with a neighbour's cluster

Challenge (fast finishers)

Add an autoscaler: `kubectl autoscale deployment hello-web --min 1 --max 5 --cpu-percent 50`, then `kubectl get hpa`. Roll out a new image with `kubectl set image deployment/hello-web hello-app=us-docker.pkg.dev/google-samples/containers/gke/hello-app:1.0` and watch `kubectl rollout status deployment/hello-web` — then roll back with `kubectl rollout undo`.

Summary

You described desired state — a Deployment of three, a Service in front — and the cluster made reality match it, including replacing a pod you deleted. The same objects, written as a manifest, applied without changing anything. And you deleted the cluster, because it bills whether or not it runs anything.

Lab 3 — An HTTP function, then a function that reacts to an upload

Estimated time	Difficulty	Environment	Kit
20 min	●●●	Cloud Shell, region asia-south1	gcp-landscape-lab3-functions.zip

OBJECTIVE

Two supplied Cloud Run functions deployed: one answering HTTP, one running whenever a file lands in a bucket — with the three IAM grants that make an Eventarc trigger work.

PREREQUISITES

- Lab 0 completed (run, cloudbuild, artifactregistry APIs enabled).
- Kit unzipped: lab3/hello-function (entry point hello) and lab3/gcs-function (entry point on_upload) — both complete, never edited.

Run everything from the lab3 folder. The first block sets three variables the later blocks use.

Steps

Step 1 — Read both functions (no edits), set the variables

CLOUD SHELL

```
cd lab3 && cat hello-function/main.py gcs-function/main.py
PROJECT=$(gcloud config get-value project)
PN=$(gcloud projects describe $PROJECT --format='value(projectNumber)')
R=asia-south1
```

Step 2 — Deploy the HTTP function and call it

Console path: Cloud Run → Write a function → name hello-function, region asia-south1, runtime Python 3.13, allow unauthenticated → paste main.py and requirements.txt → entry point hello → Deploy.

CLOUD SHELL

```
(cd hello-function && gcloud run deploy hello-function --source . --function hello --
base-image python313 --region $R --allow-unauthenticated)
curl -s "$(gcloud run services describe hello-function --region $R --
format='value(status.url)')/?name=bookshelf"; echo
```

EXPECTED

```
Service [hello-function] revision [hello-function-00001-d4x] has been deployed and is
serving 100 percent of traffic.
Service URL: https://hello-function-482915730261.asia-south1.run.app
{"from":"cloud-run-function","message":"hello, bookshelf"}
```

First deploy 2–3 minutes; answer y if asked to create the source-deploy repository.

Step 3 — Create the upload bucket

Console path: Cloud Storage → Buckets → Create → bookshelf-uploads-<project-id>, asia-south1, uniform access.

CLOUD SHELL

```
gcloud storage buckets create gs://bookshelf-uploads-$PROJECT --location=$R --uniform-
bucket-level-access
```

EXPECTED

```
Creating gs://bookshelf-uploads-sandbox-rohan-07/...
```

Step 4 — The three grants — the real lesson of event wiring

Cloud Storage events travel over Pub/Sub, so the Storage service agent must be allowed to publish (1). The trigger runs as the Compute Engine default service account, which must be allowed to receive events (2) and to invoke the function (3).

Console path: IAM & Admin → IAM → Grant access → the two principals and three roles listed here.

CLOUD SHELL

```
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:(gcloud
storage service-agent) --role=roles/pubsub.publisher
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$PN-
compute@developer.gserviceaccount.com --role=roles/eventarc.eventReceiver
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:$PN-
compute@developer.gserviceaccount.com --role=roles/run.invoker
```

EXPECTED

Updated IAM policy for project [sandbox-rohan-07]. (three times; a list of bindings follows each)

Step 5 — Deploy the event function, then create the Eventarc trigger

Console path: Cloud Run → Write a function → gcs-function, entry point on_upload, require authentication → Deploy. Then the service → Triggers → Add Eventarc trigger → Cloud Storage, google.cloud.storage.object.v1.finalized, bucket bookshelf-uploads-<project>, service account <project-number>-compute@... → Save.

CLOUD SHELL

```
(cd gcs-function && gcloud run deploy gcs-function --source . --function on_upload --
base-image python313 --region $R --no-allow-unauthenticated)
gcloud eventarc triggers create on-upload --location=$R --destination-run-service=gcs-
function --destination-run-region=$R \
--event-filters="type=google.cloud.storage.object.v1.finalized" --event-
filters="bucket=bookshelf-uploads-$PROJECT" \
--service-account=$PN-compute@developer.gserviceaccount.com
```

EXPECTED

Service [gcs-function] revision [gcs-function-00001-h2k] has been deployed ...
Creating trigger [on-upload] in project [sandbox-rohan-07], location [asia-south1]...done.
WARNING: It may take up to 2 minutes for the new trigger to become active.

CHECK YOURSELF

- Two services in gcloud run services list: hello-function (public) and gcs-function (authenticated).
- gcloud eventarc triggers list shows on-upload with destination gcs-function.

Step 6 — Upload a file, read the log line

Console path: Cloud Storage → bucket → Upload files; then Cloud Run → gcs-function → Logs.

CLOUD SHELL

```
echo hello > note.txt; gcloud storage cp note.txt gs://bookshelf-uploads-$PROJECT/;
sleep 30
gcloud run services logs read gcs-function --region $R --limit 5
```

EXPECTED

```
2026-09-07 11:02:44 new object: gs://bookshelf-uploads-sandbox-rohan-07/note.txt
size=6 bytes type=text/plain
2026-09-07 11:02:44 event id=12345678901234
type=google.cloud.storage.object.v1.finalized
2026-09-07 11:02:44 POST 200 41ms /
```

Nothing after 30 s? The first trigger can take up to 2 minutes to become active — upload once more and read again.

Step 7 — Teardown — trigger, functions, bucket**CLOUD SHELL**

```
gcloud eventarc triggers delete on-upload --location=$R --quiet
gcloud run services delete gcs-function --region $R --quiet; gcloud run services
delete hello-function --region $R --quiet
gcloud storage rm -r gs://bookshelf-uploads-$PROJECT
gcloud run services list --region $R          # must be empty
```

Troubleshooting

Symptom	Cause	Fix
No log line after two uploads	One of the three grants missing, or the trigger not yet active	gcloud eventarc triggers describe on-upload — an error names the missing role; wait 2 min
triggers create: permission denied on service account	Grant 2/3 not applied or wrong project number	echo \$PN; re-run the two bindings
deploy asks to enable an API	Cloud Build / Artifact Registry off	Answer y (or Lab 0 step 3)
HTTP function 403	Deployed without --allow-unauthenticated	Re-deploy with the flag, or add allUsers as Cloud Run Invoker

Challenge (fast finishers)

Upload the same file again and compare the two event ids in the log — same object, two events. Then create a second trigger on `google.cloud.storage.object.v1.deleted` pointing at the same function, delete `note.txt` from the bucket, and read what the handler printed.

Summary

An HTTP function is a Cloud Run service you never containerised. An event function is the same thing wired to a producer through Eventarc — three permissions and one filter — and a file landing in a bucket ran your code with no server anywhere. The handler may run twice for one event; design for it.

Lab 4 — A private subnet that can still reach Google

Estimated time	Difficulty	Environment	Kit
20 min	...	Cloud Shell, region asia-south1	gcp-landscape-lab4-starter.zip (solution: -solution.zip)

OBJECTIVE

A custom-mode VPC with one planned subnet, an SSH-from-IAP-only firewall rule, a VM with no external IP, and Private Google Access proven to work — then removed in the right order.

PREREQUISITES

- Lab 0 completed.
- Kit unzipped: lab4.sh with four blanks, teardown.sh.
- First `gcloud compute ssh` in Cloud Shell generates an SSH key: accept the prompts (empty passphrase is fine in a sandbox).

Steps

Step 1 — Create a custom-mode VPC

Console path: VPC network → VPC networks → Create VPC network → name bookshelf-vpc → Subnet creation mode: Custom → Create (no subnets yet).

CLOUD SHELL

```
gcloud compute networks create bookshelf-vpc --subnet-mode=custom
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/global/networks/bookshelf-vpc].
NAME          SUBNET_MODE  BGP_ROUTING_MODE  IPV4_RANGE  GATEWAY_IPV4
bookshelf-vpc  CUSTOM      REGIONAL          10.10.0.0/24
```

Step 2 — Fill blank 4.2 — one subnet with a /24 inside 10.0.0.0/8

A /24 is 256 addresses; Google reserves four (network, gateway, second-to-last, broadcast) so 252 are usable.

Console path: bookshelf-vpc → Subnets → Add subnet → name bookshelf-private, region asia-south1, IPv4 range 10.10.0.0/24.

CLOUD SHELL

```
gcloud compute networks subnets create bookshelf-private \
  --network=bookshelf-vpc --region=asia-south1 --range=10.10.0.0/24
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/regions/asia-south1/subnetworks/bookshelf-private].
NAME          REGION      NETWORK          RANGE          STACK_TYPE
bookshelf-private  asia-south1  bookshelf-vpc    10.10.0.0/24   IPV4_ONLY
```

Step 3 — Fill blank 4.3 — SSH only from Google's IAP TCP-forwarding range

35.235.240.0/20 is the fixed range Identity-Aware Proxy uses for TCP forwarding. Nothing else is open on this network.

Console path: VPC network → Firewall → Create firewall rule → name bookshelf-allow-iap-ssh, network bookshelf-vpc, ingress, allow, target tag private-vm, source 35.235.240.0/20, tcp:22.

CLOUD SHELL

```
gcloud compute firewall-rules create bookshelf-allow-iap-ssh \
```

```
--network=bookshelf-vpc --direction=INGRESS --action=ALLOW \
--rules=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=private-vm
```

EXPECTED

Creating firewall...done.

NAME	NETWORK	DIRECTION	PRIORITY	ALLOW	DENY	DISABLED
bookshelf-allow-iap-ssh	bookshelf-vpc	INGRESS	1000	tcp:22		False

Step 4 — Look at the routes the VPC got for free

Console path: VPC network → Routes → filter by network bookshelf-vpc.

CLOUD SHELL

```
gcloud compute routes list --filter="network:bookshelf-vpc"
```

EXPECTED

NAME	NETWORK	DEST_RANGE	NEXT_HOP
default-route-3f1a9c0b2e7d4a61	bookshelf-vpc	10.10.0.0/24	bookshelf-vpc
default-route-9b8e2d4c6a1f0e37	bookshelf-vpc	0.0.0.0/0	default-internet-gateway

Step 5 — Fill blank 4.5 — a VM with NO external IP

Console path: Compute Engine → Create instance → e2-micro, Debian 12 → Networking: tag private-vm, network interface bookshelf-vpc / bookshelf-private, External IPv4 address: None.

CLOUD SHELL

```
gcloud compute instances create private-vm --zone=asia-south1-a \
--machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud \
--subnet=bookshelf-private --no-address --tags=private-vm
```

EXPECTED

NAME	ZONE	MACHINE_TYPE	PREEMPTIBLE	INTERNAL_IP	EXTERNAL_IP	STATUS
private-vm	asia-south1-a	e2-micro		10.10.0.2		RUNNING

Step 6 — SSH in through IAP and prove nothing is reachable (yet)

Both curls time out after 5 seconds. That is the expected result: no external IP means no way out.

Console path: Compute Engine → VM instances → private-vm → SSH (the console button also tunnels through IAP).

CLOUD SHELL

```
gcloud compute ssh private-vm --zone=asia-south1-a --tunnel-through-iap \
--command='curl -s -m 5 -o /dev/null -w "internet: HTTP %{http_code}\n"
https://example.com || echo "internet: unreachable (expected)"; curl -s -m 5 -o
/dev/null -w "storage api: HTTP %{http_code}\n" https://storage.googleapis.com || echo
"storage api: unreachable (expected)";'
```

EXPECTED

```
internet: unreachable (expected)
storage api: unreachable (expected)
```

CHECK YOURSELF

- The VM has an INTERNAL_IP and an empty EXTERNAL_IP column.
- SSH through IAP works (the firewall rule is what allows it).

- Both curls print unreachable.

Step 7 — Fill blank 4.7 — turn on Private Google Access

Console path: VPC network → bookshelf-vpc → Subnets → bookshelf-private → Edit → Private Google Access: On → Save.

CLOUD SHELL

```
gcloud compute networks subnets update bookshelf-private --region=asia-south1 \
--enable-private-ip-google-access
```

EXPECTED

```
Updated [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/regions/asia-south1/subnetworks/bookshelf-private].
```

Step 8 — Prove Google APIs now work and the internet still does not

HTTP 400 from storage.googleapis.com is success: the API answered (400 just means the request named no bucket).

CLOUD SHELL

```
gcloud compute ssh private-vm --zone=asia-south1-a --tunnel-through-iap \
--command='curl -s -m 5 -o /dev/null -w "internet: HTTP %{http_code}\n"
https://example.com || echo "internet: unreachable (expected)"; curl -s -m 5 -o
/dev/null -w "storage api: HTTP %{http_code}\n" https://storage.googleapis.com; gcloud
storage ls 2>&1 | head -3'
```

EXPECTED

```
internet: unreachable (expected)
storage api: HTTP 400
gs://bookshelf-uploads-sandbox-rohan-07/
```

If the storage line still says unreachable, wait 30 seconds and re-run — the subnet change takes a few seconds to propagate.

Step 9 — Tear down — in order

VM first, then the firewall rule, then the subnet, then the network. A subnet with a VM in it cannot be deleted.

CLOUD SHELL

```
bash teardown.sh
gcloud compute networks list # only 'default' remains
```

EXPECTED

```
Lab 4 network removed.
NAME      SUBNET_MODE  BGP_ROUTING_MODE  IPV4_RANGE  GATEWAY_IPV4
default  AUTO        REGIONAL
```

Troubleshooting

Symptom	Cause	Fix
gcloud compute ssh hangs	IAP firewall rule missing or wrong source range	Rule must allow tcp:22 from 35.235.240.0/20 to tag private-vm
"Permission denied" on SSH	Key generation prompt was cancelled	Re-run; accept the key prompts
Step 8 storage line still unreachable	PGA not yet propagated, or wrong subnet	Wait 30 s; check subnets describe shows privateIpGoogleAccess: true

Symptom	Cause	Fix
Subnet delete fails	VM still exists	teardown.sh deletes in order; re-run it

Challenge (fast finishers)

Give the VM internet access without an external IP: create a Cloud Router (`gcloud compute routers create nat-router --network=bookshelf-vpc --region=asia-south1`) and Cloud NAT (`gcloud compute routers nats create nat-config --router=nat-router --region=asia-south1 --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges`), then re-run step 8 and watch the internet line change. Delete both before teardown.

Summary

Custom VPC, planned range, narrow firewall, no external IP, Private Google Access: that is the shape of every production subnet. The best news of the lab was an HTTP 400 — it proved the private path works.

Lab 5 — Postgres from psql, then Firestore documents

Estimated time	Difficulty	Environment	Kit
25 min (+ 5–10 min instance creation at kick-off)	●●○	Cloud Shell, region asia-south1	none — typed and clicked

OBJECTIVE

A Cloud SQL PostgreSQL instance with a table you created and queried from psql in Cloud Shell, an on-demand backup, and a Firestore database with documents you added in the console and queried from Cloud Shell.

PREREQUISITES

- Lab 0 completed.
- The Cloud SQL instance creation started at the chapter kick-off (Step 0 below).
- No kit — everything is typed or clicked.

Steps

Step 0 — Kick-off (done at the start of the chapter) — create the Cloud SQL instance

Five to ten minutes. Shared-core db-f1-micro, Enterprise edition, PostgreSQL 17. The password is the lab's and obviously not a real one.

Console path: SQL → Create instance → PostgreSQL → instance id bookshelf-pg, password Bookshelf#2026, region asia-south1, single zone, Enterprise edition, Sandbox (shared core) preset → Create.

CLOUD SHELL

```
gcloud services enable sqladmin.googleapis.com firestore.googleapis.com
gcloud sql instances create bookshelf-pg --database-version=POSTGRES_17 --tier=db-f1-micro \
  --region=asia-south1 --edition=ENTERPRISE --root-password='Bookshelf#2026'
```

EXPECTED

```
Creating Cloud SQL instance for POSTGRES_17...done.
NAME          DATABASE_VERSION  LOCATION          TIER          PRIMARY_ADDRESS
PRIVATE_ADDRESS STATUS
bookshelf-pg  POSTGRES_17      asia-south1-b    db-f1-micro   34.93.77.140    -
RUNNABLE
```

Step 1 — Create the bookshelf database and connect with psql

gcloud sql connect allowlists Cloud Shell's public IP on the instance for five minutes and opens psql (installed in Cloud Shell). Production would use private IP and the Cloud SQL Auth Proxy instead — never a permanent open authorized network.

Console path: SQL → bookshelf-pg → Databases → Create database → bookshelf. For a browser psql: SQL → bookshelf-pg → Cloud SQL Studio → sign in as postgres.

CLOUD SHELL

```
gcloud sql databases create bookshelf --instance=bookshelf-pg
gcloud sql connect bookshelf-pg --user=postgres --database=bookshelf
```

EXPECTED

```
Created database [bookshelf].
Allowlisting your IP for incoming connection for 5 minutes...done.
Password: (Bookshelf#2026)
psql (17.x) ... SSL connection
bookshelf=>
```

Step 2 — Inside psql — create the table, insert four rows, query, quit

Type at the bookshelf=> prompt. Relational: the schema comes first, then the rows.

CLOUD SHELL

```
CREATE TABLE books (id serial PRIMARY KEY, title text NOT NULL, author text, year
int);
INSERT INTO books (title, author, year) VALUES
('The Pragmatic Programmer','Hunt & Thomas',1999),
('Designing Data-Intensive Applications','Kleppmann',2017),
('Site Reliability Engineering','Beyer et al.',2016),
('Release It!','Nygard',2018);
SELECT title, year FROM books WHERE year >= 2017 ORDER BY year;
\q
```

EXPECTED

```
CREATE TABLE
INSERT 0 4
          title                      | year
-----+-----
Designing Data-Intensive Applications | 2017
Release It!                          | 2018
(2 rows)
```

Step 3 — An on-demand backup, and where the instance lives

Console path: SQL → bookshelf-pg → Backups → Create backup. Overview tab shows the public IP, zone and machine.

CLOUD SHELL

```
gcloud sql backups create --instance=bookshelf-pg
gcloud sql instances describe bookshelf-pg --
format='value(ipAddresses[0].ipAddress,gceZone,settings.tier) '
```

EXPECTED

```
Backing up Cloud SQL instance...done.
34.93.77.140  asia-south1-b  db-f1-micro
```

Automated daily backups and point-in-time recovery are the production settings; this on-demand backup is in addition.

CHECK YOURSELF

- The SELECT returned Kleppmann (2017) and Nygard (2018).
- The backup shows as SUCCESSFUL under SQL → bookshelf-pg → Backups.

Step 4 — Firestore — create the database, add two documents in the console

One database per project by default, named (default), Native mode, regional. Documents are added by hand here so you see that no schema exists: give book-2 a field book-1 does not have.

Console path: Firestore → Create database → Native mode, location asia-south1 → Create. Then Data → Start collection → collection id books → document id book-1, fields title (string) Release It!, author (string) Nygard, year (number) 2018 → Save. Add document book-2: title (string) SRE, year (number) 2016, free (boolean) true.

CLOUD SHELL

```
gcloud firestore databases create --location=asia-south1 --type=firestore-native
```

EXPECTED

```
Waiting for [operations/...] to finish...done.
Created database [(default)].
```

Step 5 — Read and query the documents from Cloud Shell (REST — the same calls the client libraries make)**CLOUD SHELL**

```
PROJECT=$(gcloud config get-value project)
TOKEN=$(gcloud auth print-access-token)
FS="https://firestore.googleapis.com/v1/projects/$PROJECT/databases/(default)/documents"
curl -s -H "Authorization: Bearer $TOKEN" "$FS/books" | head -30
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
"$FS:runQuery" \
-d
'{"structuredQuery":{"from":[{"collectionId":"books"}],"where":{"fieldFilter":{"field":{"fieldPath":"year"},"op":"GREATER_THAN_OR_EQUAL","value":{"integerValue":"2017"}}}}}'
```

EXPECTED

```
{"documents": [ {"name": "projects/sandbox-rohan-07/databases/(default)/documents/books/book-1", "fields": {"title": {"stringValue": "Release It!"}, ... [{"document":{"name":".../documents/books/book-1","fields":{"title":{"stringValue":"Release It!"},"author":{"stringValue":"Nygard"},"year":{"integerValue":"2018"}}}, ...]}
```

A single-field filter needs no index work; a query combining two fields would fail with a message that links to the composite index to create.

Step 6 — Teardown — the instance and the database

Console path: SQL → bookshelf-pg → Delete. Firestore → (default) → Delete database (disable delete protection first if the console asks).

CLOUD SHELL

```
gcloud sql instances delete bookshelf-pg --quiet
gcloud firestore databases delete --database='(default)' --quiet
gcloud sql instances list # must be empty
```

EXPECTED

```
Deleted [https://sqladmin.googleapis.com/sql/v1beta4/projects/sandbox-rohan-07/instances/bookshelf-pg].
Listed 0 items.
```

Troubleshooting

Symptom	Cause	Fix
connect: instance not RUNNABLE / times out	Still creating	gcloud sql instances list and wait; use Cloud SQL Studio meanwhile
psql: password authentication failed	Typo in the kick-off password	gcloud sql users set-password postgres --instance=bookshelf-pg --password='Bookshelf#2026'
relation "books" already exists	CREATE TABLE re-run	Carry on — the rows are there; SELECT to confirm
runQuery returns 400	Quoting around \$FS or the JSON body	Paste the block exactly; the (default) segment must stay inside the quotes
firestore databases create: already exists	The project had a database	Skip to step 4's console part

Challenge (fast finishers)

Add a subcollection: under book-1 start a collection reviews with a document {stars: 5, text: "..."} and read it with curl at \$FS/books/book-1/reviews. On the Postgres side, enable automated backups with point-in-time recovery: `gcloud sql instances patch bookshelf-pg --backup-start-time=02:00 --enable-point-in-time-recovery`, then describe the instance and find the setting.

Optional extra — Memorystore (needs a VM in the VPC)

Memorystore is reachable only by private IP, so Cloud Shell cannot talk to it; this extra uses a temporary VM in the default network. Create: `gcloud redis instances create bookshelf-cache --size=1 --region=asia-south1 --network=default` (about five minutes). Read its IP: `gcloud redis instances describe bookshelf-cache --region=asia-south1 --format='value(host)'`. Create a VM: `gcloud compute instances create cache-client --zone=asia-south1-a --machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud`, SSH in, `sudo apt-get install -y redis-tools`, then `redis-cli -h <host> SET book:1 'Release It!' EX 300` and `redis-cli -h <host> GET book:1`. Teardown: delete the VM and `gcloud redis instances delete bookshelf-cache --region=asia-south1 --quiet`.

Summary

Two managed databases, two shapes. Cloud SQL gave you PostgreSQL with backups and a connectivity decision you saw up close — the five-minute allowlist is the wrong way for production and the reason the Auth Proxy exists. Firestore gave you documents with no schema and a query that needed no index — and a bill that counts reads, so design the app to read less.

Activities A and B (no keyboard)

Activity A — The decision matrix

Format: pairs, about 12 minutes. Copy the grid below into a notebook. For each workload card, tick VM / GKE / Cloud Run-or-function per criterion, then write the decision and the one criterion that decided it.

Criterion	Card 1 · Order-export job	Card 2 · Licensed reporting engine	Card 3 · Public catalogue API	Card 4 · Bookshelf web + api
1 State	stateless — CR job ✓ (worked example)			
2 Traffic shape				
3 Latency budget				
4 Control needed				
5 Team				
6 Portability				
Decision + deciding criterion				

Card 1 — runs nightly at 02:00 for ~15 min; reads the orders DB; writes a CSV to a bucket. Card 2 — vendor binary needing a kernel module and a static licence IP; 20 analysts 9–6. Card 3 — 0 req/s at night, 400 req/s at 9 am; < 300 ms; stateless; no ops team. Card 4 — two small stateless Python services; web public, api reachable by web only; the team is this room.

MODEL ANSWERS (TRAINER RELEASES AFTER THE DEBRIEF)

- Card 1 → Cloud Run job on Cloud Scheduler — traffic shape decides.
- Card 2 → Compute Engine VM in a MIG of 1 — control decides.
- Card 3 → Cloud Run service with min-instances 1–2 — latency and team decide.
- Card 4 → two Cloud Run services, api internal + IAM-only — state and team decide (the container session builds it).

Activity B — Pick the store

Format: individually, then compare with a neighbour, about 6 minutes. Use the decision tree from the deck: files or records → transactional or analytical → relational or not → one region or many.

#	Workload	Your pick	Why (one line)
1	Product images uploaded by sellers, served on every page view, kept forever		
2	Orders and payments for one country; strict consistency; 2 000 writes/s at peak		
3	"Revenue by category by hour, last 3 years" for the finance dashboard		
4	Per-user shopping carts with live updates on web and mobile		
5	Bookshelf's catalogue — 4 books today, maybe 4 000 next year; read by the api on every request		

MODEL ANSWERS

- 1 → Cloud Storage, Standard (multi-region if global) + Cloud CDN
- 2 → Cloud SQL (PostgreSQL); Spanner only for multi-region writes
- 3 → BigQuery
- 4 → Firestore
- 5 → Cloud SQL (+ Memorystore later); today's in-memory list is the placeholder

Optional — a catalogue bucket that cleans up after itself

Estimated time	Difficulty	Environment	Kit
15 min	●●○	Cloud Shell, region asia-south1	none (optional)

OBJECTIVE

A regional Standard bucket holding the catalogue exports, with lifecycle rules, versioning and an (unlocked) retention policy — then removed.

PREREQUISITES

- Lab 0 completed.
- Optional extra for fast groups or self-study: no kit in v2 — type the lifecycle.json shown in chapter 6 into a file (python3 -m json.tool checks it) and create three small text files as docs/.

Steps

Step 1 — Create the bucket: regional, Standard, uniform bucket-level access

Bucket names are global across Google Cloud; the script appends your project id to keep yours unique.

Console path: Cloud Storage → Buckets → Create → name bookshelf-docs-<your-project-id> → Region asia-south1 → Standard → Uniform access → Create.

CLOUD SHELL

```
PROJECT=$(gcloud config get-value project)
BUCKET=gs://bookshelf-docs-$PROJECT
gcloud storage buckets create $BUCKET --location=asia-south1 \
  --default-storage-class=STANDARD --uniform-bucket-level-access
```

EXPECTED

```
Creating gs://bookshelf-docs-sandbox-rohan-07/...
```

Step 2 — Upload the exports and list them

Console path: Bucket → Upload files → select docs/*.txt (put them in a folder named exports).

CLOUD SHELL

```
gcloud storage cp docs/*.txt $BUCKET/exports/
gcloud storage ls -l $BUCKET/exports/
```

EXPECTED

```
Copying file://docs/catalogue-1.txt to gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt
...
 231  2026-09-06T05:31:12Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt
 231  2026-09-06T05:31:12Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-2.txt
 231  2026-09-06T05:31:13Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-3.txt
```

Step 3 — Fill the three blanks in lifecycle.json, validate, apply

Rule 1: Standard objects move to NEARLINE after 30 days. Rule 2: an old version is deleted once 3 newer versions exist. Rule 3: everything is deleted at 365 days.

Console path: Bucket → Lifecycle tab → Add a rule (three times) with the same action/condition pairs.

CLOUD SHELL

```
python3 -m json.tool lifecycle.json > /dev/null && echo valid
gcloud storage buckets update $BUCKET --lifecycle-file=lifecycle.json
gcloud storage buckets describe $BUCKET --format="yaml(lifecycle_config)"
```

EXPECTED

```
valid
Updating gs://bookshelf-docs-sandbox-rohan-07/...
lifecycle_config:
  rule:
  - action: {storageClass: NEARLINE, type: SetStorageClass}
    condition: {age: 30, matchesStorageClass: [STANDARD]}
  - action: {type: Delete}
    condition: {isLive: false, numNewerVersions: 3}
  - action: {type: Delete}
    condition: {age: 365}
```

Answers: "NEARLINE", 3, 365. Rules evaluate about once a day; transitions are asynchronous.

Step 4 — Turn on versioning, overwrite a file, list the generations

Console path: Bucket → Protection tab → Object versioning: On.

CLOUD SHELL

```
gcloud storage buckets update $BUCKET --versioning
echo "id,title,author,year" > docs/catalogue-1.txt
gcloud storage cp docs/catalogue-1.txt $BUCKET/exports/catalogue-1.txt
gcloud storage ls -a $BUCKET/exports/catalogue-1.txt
```

EXPECTED

```
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145601234567
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145688765432
```

CHECK YOURSELF

- describe shows three lifecycle rules.
- ls -a shows two generations of catalogue-1.txt (the # suffix is the generation number).

Step 5 — Set a 1-day retention policy, then try to delete an object

The delete is expected to fail — that is the feature. The policy is NOT locked, so teardown can clear it. Never lock a retention policy on a sandbox bucket.

Console path: Bucket → Protection tab → Retention policy → Set → 1 day (do NOT lock).

CLOUD SHELL

```
gcloud storage buckets update $BUCKET --retention-period=P1D
gcloud storage rm $BUCKET/exports/catalogue-2.txt || echo "--> blocked by retention, as expected"
```

EXPECTED

```
ERROR: (gcloud.storage.rm) HTTPError 403: Object 'bookshelf-docs-sandbox-rohan-07/exports/catalogue-2.txt' is subject to bucket's retention policy and cannot be deleted, overwritten or archived until 2026-09-07T... --> blocked by retention, as expected
```

Step 6 — Move one object to ARCHIVE by hand

This is what the lifecycle rule would do for you later; doing it once shows the class is per object.

Console path: Object → Edit storage class → Archive.

CLOUD SHELL

```
gcloud storage objects update $BUCKET/exports/catalogue-3.txt --storage-class=ARCHIVE
gcloud storage ls -L $BUCKET/exports/catalogue-3.txt | grep -i "storage class"
```

EXPECTED

```
Storage Class:  ARCHIVE
```

Step 7 — Tear down

Tear down clears the (unlocked) retention policy first, then deletes every version, then the bucket.

CLOUD SHELL

```
bash teardown.sh
gcloud storage ls      # your bucket must be gone
```

EXPECTED

```
Lab 3 bucket removed.
```

Troubleshooting

Symptom	Cause	Fix
409 "already exists" on create	Bucket names are global; the project-id suffix was lost	Check BUCKET=gs://bookshelf-docs-\$PROJECT printed your project id
lifecycle-file rejected	JSON typo (quotes, trailing comma)	python3 -m json.tool lifecycle.json shows the line
Tear down says retention policy still active	Policy was locked	It cannot be undone — leave the bucket; it expires in 1 day
rm succeeded in step 5	Retention step skipped or ran after rm	Re-run step 5 in order

Challenge (fast finishers)

Enable Autoclass on a fresh bucket (`gcloud storage buckets create gs://bookshelf-auto-$PROJECT --location=asia-south1 --enable-autoclass --uniform-bucket-level-access`) and compare its describe output with the rule-based bucket. Which would you choose for the catalogue exports, and why? Delete it afterwards.

Summary

Class, lifecycle, versioning, retention — four settings, and the bucket now enforces the data policy you used to write in a wiki. The one thing to remember: a locked retention policy is permanent.

Appendix — Environment and versions

Item	Pinned	Note
gcloud CLI	583.x	Cloud Shell keeps itself current; re-verify with gcloud --version
Region / zone	asia-south1 / asia-south1-a	Mumbai — all labs
VM image	debian-12 (debian-cloud)	image family, not a dated image
Machine types	e2-micro (VMs), e2-small (MIG template)	quota-friendly
GKE	Autopilot, regional (asia-south1), sample image us-docker.pkg.dev/google-samples/containers/gke/hello-app:2.0	kubectl pre-installed in Cloud Shell
Cloud Run functions	gcloud run deploy --function · base image python313 · functions-framework 3.10.2, cloudevents 1.12.0	Cloud Functions is now Cloud Run functions; GCS events via Eventarc
Cloud SQL	PostgreSQL 17, Enterprise, db-f1-micro, asia-south1; psql from Cloud Shell via gcloud sql connect	public IP + 5-minute allowlist in the lab; private IP + Auth Proxy in production
Firestore	Native mode, (default) database, asia-south1; REST API from Cloud Shell	console for adding documents
Cloud Run services / Artifact Registry	—	taught in the container session, not here

APIs enabled in Lab 0

- compute.googleapis.com
- run.googleapis.com
- cloudbuild.googleapis.com
- artifactregistry.googleapis.com
- secretmanager.googleapis.com
- storage.googleapis.com
- container.googleapis.com (enabled at the Lab 2 kick-off)
- eventarc.googleapis.com (enabled by the first trigger create)
- sqladmin.googleapis.com and firestore.googleapis.com (enabled at the Lab 5 kick-off)

Kits

Lab	Starter	Solution
1	gcp-landscape-lab1-starter.zip — lab1.sh (3 blanks), startup.sh, teardown.sh	gcp-landscape-lab1-solution.zip — completed lab1.sh
2	gcp-landscape-lab2-starter.zip — bookshelf-api/, hello-function/, lab2.sh (4 blanks), teardown.sh	completed lab2.sh
3	gcp-landscape-lab3-starter.zip — docs/, lifecycle.json (3 blanks), lab3.sh, teardown.sh	completed lifecycle.json
4	gcp-landscape-lab4-starter.zip — lab4.sh (4 blanks), teardown.sh	completed lab4.sh

Lab	Starter	Solution
5	gcp-landscape-lab5-starter.zip — api/, web/, deploy.sh (4 blanks), teardown.sh	completed deploy.sh

End-of-session teardown order

- Lab 2: `kubectl delete service hello-web` · `gcloud container clusters delete hello-cluster --location asia-south1 --quiet`
- Lab 5: `gcloud sql instances delete bookshelf-pg --quiet` · `gcloud firestore databases delete --database='(default)' --quiet`
- Lab 3: `gcloud eventarc triggers delete on-upload` · `gcloud run services delete gcs-function / hello-function` · `gcloud storage rm -r gs://bookshelf-uploads-<project>`
- Lab 4: `bash lab4/teardown.sh` (VM → firewall → subnet → network)
- Lab 1: `bash lab1/teardown.sh` if skipped earlier
- Verify: `gcloud compute instances list` · `gcloud container clusters list` · `gcloud run services list` · `gcloud sql instances list` · `gcloud storage ls` · `gcloud compute networks list` → only 'default' remains

Volatile facts (versions, product names, console navigation) were verified against docs.cloud.google.com on 6 September 2026. Re-verify before each delivery.